

67.101 - 67.130
12/10/01

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדוראלי

תאריך: 11.7.2008
משך הבחינה: שלוש שעות

מועד ב' - סמסטר א' - תשס"ח – 2007-2008
המורה: פרופ' ג'ף רוזנשיין

הנחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיוטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
- פתרון מיטבי לשאלה יחשב פתרון **פשוט ויעיל ככל האפשר**. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות** או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
- בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0 אלא אם מבוהר בשאלה אחרת.

שאלה 1 (33 נקודות)

נתונה המחלקה Node, אותה כבר מימשנו עבורכם ובה השיטות:

```
public Node getNext()
public void setNext(Node next)
```

א. (17 נקודות)

כיתבו שיטה `public static Node reverseList(Node head)` המקבלת מצביע לראש רשימה חד-כיוונית חסרת מעגלים, הופכת את הרשימה ומחזירה את ראש הרשימה ההפוכה.

שימו לב:

- ניתן להקצות רק זיכרון בגודל קבוע (כלומר שאינו תלוי במספר ה-nodes ברשימה).
- על השיטה לעבוד בסיבוכיות זמן של $O(n)$ (כאשר n מספר ה-nodes ברשימה). הראו כי עמדתם בדרישה זו.

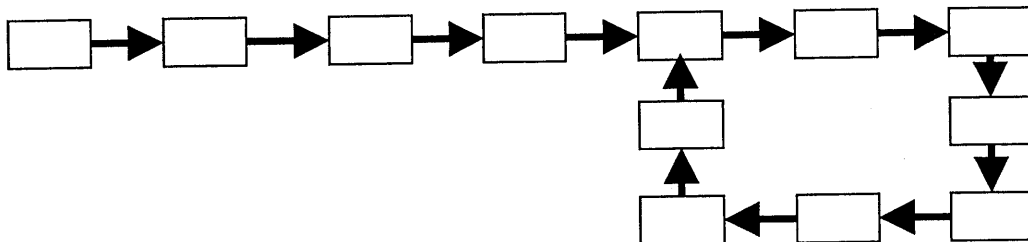
ב. (16 נקודות)

כיתבו שיטה `public static void fixCycle(Node head, Node innerNode)` המקבלת מצביע לראש רשימה המכילה מעגל ומצביע לחוליה כלשהי בתוך המעגל והופכת את הרשימה לרשימה חסרת מעגלים תוך שמירה על סדר האיברים.

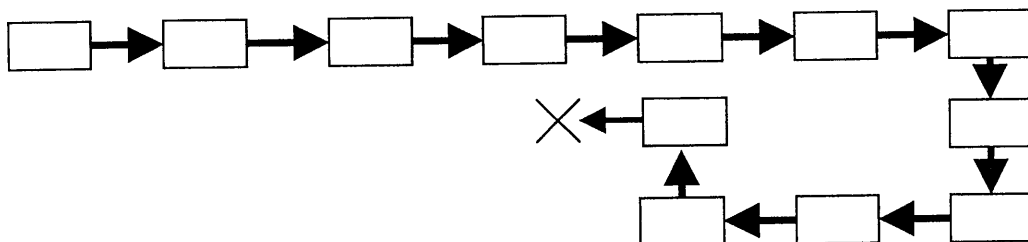
שימו לב:

- ניתן להקצות רק זיכרון בגודל קבוע.
- במידת הצורך ניתן להגדיר ולממש שיטות עזר.
- ניתן להשתמש ב- `reverseList()` מהסעיף הקודם (גם אם לא פתרתם סעיף זה).
- על השיטה לעבוד בסיבוכיות זמן של $O(n)$. הראו כי עמדתם בדרישה זו.

דוגמה לרשימה המכילה מעגל:



אותה הרשימה לאחר `fixCycle`:



67. 101 - 67. 130
 101 / 67

שאלה 2 (33 נקודות)

אין לפתור שאלה זו ברקורסיה.

נתון מערך $A[1...n]$ המכיל מספרים שלמים שונים זה מזה.

נגדיר נקודת שבת במערך כאינדקס i שעבורו $A[i] = i$.

א. (7 נקודות)

כתוב שיטה `public static int findShevet (int[ ] A)`

המקבלת את המערך A ומחזירה את האינדקס i אם קיימת במערך נקודת שבת, אחרת על השיטה להחזיר את הערך -1 . (אם קיימת יותר מנקודת שבת אחת, על השיטה להחזיר אחת מהן). על השיטה לעבוד במקרה הגרוע בזמן $O(n)$.

ב. (18 נקודות)

נניח כעת כי המערך A ממויין בסדר עולה. שפר את השיטה שכתבת בסעיף א' כך שניתן לחסום מלעיל את סבוכיות זמן הריצה במקרה הגרוע בחסם נמוך יותר מ- $O(n)$.

ג. (8 נקודות)

מהי סיבוכיות זמן הריצה של השיטה שכתבת בסעיף ב' במקרה הגרוע ביותר, הסבר? מהי סיבוכיות זמן הריצה של השיטה שכתבת בסעיף ב' במקרה הטוב ביותר, הסבר?

שאלה 3 (33 נקודות)

רקורסיה: הפירמידות של מונטזומה.

נתונה המחלקה `Stack` שמממשת מחסנית המחזיקה נתונים מטיפוס `int`.

המחסנית מאחסנת נתונים במדיניות Last-In-First-Out.

המחלקה כבר ממומשת ואפשר להשתמש בשיטות שלה:

<code>public Stack()</code>	בנאי (Constructor) שיוצר מחסנית חדשה ריקה מאיברים.
<code>public void push(int num)</code>	הכנסת המספר <code>num</code> לראש המחסנית.
<code>public int pop()</code>	הוצאת המספר הנמצא בראש המחסנית והחזרתו. שיטה זו תגרום לשגיאה בתוכנית אם המחסנית ריקה בעת הקריאה לה.
<code>public int peek()</code>	החזרת ערכו של המספר הנמצא בראש המחסנית ללא הסרתו מהמחסנית. שיטה זו תגרום לשגיאה בתוכנית אם המחסנית ריקה בעת הקריאה לה.
<code>public boolean isEmpty()</code>	החזרת הערך <code>true</code> אם המחסנית ריקה. אחרת החזרת הערך <code>false</code> .

67-101-67-130
12/40P1

א. (10 נקודות)

כתבו שיטה סטטית המקבלת מחסנית, ומדפיסה את התוכן שלה כך שהאיבר שהוכנס ראשון למחסנית יודפס ראשון (שימו לב: סדר זה הפוך מסדר ההוצאה מהמחסנית). לאחר הדפסת כל מספר במחסנית יש להדפיס רווח. שימו לב: המחסנית צריכה להישאר ללא שינוי בסיום ההדפסה (כלומר בסוף ההדפסה עליה להכיל את אותם המספרים באותו הסדר). בעת מימוש השיטה אסור להקצות זיכרון נוסף על ה-heap (למשל אסור להקצות מערכים או אובייקט מחסנית נוסף). חתימת השיטה שעליכם לממש היא:

```
public static void printStack(Stack stack)
```

דוגמה: אם למחסנית הוכנס תחילה המספר 8, לאחריו הוכנס המספר 5 ולבסוף הוכנס המספר 2, יש להדפיס (משמאל לימין): 8 5 2
אין לרדת שורה בסוף ההדפסה. עבור מחסנית ריקה אין להדפיס כלום.

ב. (23 נקודות)

מונטזומה מלך האצטקים אוהב לבנות פירמידות. לרשותו של מונטזומה עומדות N אבנים ענקיות בגדלים $\{N, \dots, 2, 1\}$ – בדיוק אבן אחת מכל גודל. כדי שהפירמידות תהיינה יציבות להניח אבן רק על גבי אבן גדולה ממנה או על הקרקע. כלומר, את האבן בגודל N מותר להניח רק על הקרקע. אבן בגודל $N-1$ מותר להניח על הקרקע, או על האבן בגודל N . אבן בגודל 1 מותר להניח על כל אבן אחרת או על הקרקע. מונטזומה תמיד משתמש בכל האבנים שברשותו, והוא תמיד מניח רק אבן אחת בכל קומה של פירמידה כלשהיא.

כתבו שיטה המדפיסה את כל האפשרויות לבנית פירמידות מהאבנים של מונטזומה.

```
public static void printAllPyramids(int numBlocks)
```

כל פירמידה תודפס בתוך סוגריים מרובעים [], החל מגודל האבן שבבסיסה ומעלה (אין צורך לדאוג לרווחים מיותרים בתוך הסוגריים המרובעים אך יש לוודא שכל המספרים מופרדים ע"י רווח אחד לפחות). בסיום ההדפסה של אפשרות מסוימת לבניית הפירמידות יש לרדת שורה. סדר ההדפסה של הפירמידות השונות באותה תצורה אינו חשוב. שתי תצורות המכילות את אותן פירמידות בסדר שונה יחשבו לזהות (ואין להדפיסן פעמיים). גם סדר ההדפסות של תצורות שונות אינו חשוב כל עוד מודפסות כל התצורות.

דוגמה: הקריאה (3) printAllPyramids תדפיס את תצורות הפירמידות הבאות (לאו דווקא בסדר הזה):

```
[ 3 2 1 ]  
[ 3 2 ] [ 1 ]  
[ 3 1 ] [ 2 ]  
[ 3 ] [ 2 1 ]  
[ 3 ] [ 2 ] [ 1 ]
```

מותר להיעזר במחלקת המחסנית שהוגדרה בסעיף א', וגם בשיטה שכתבתם בסעיף ההוא (הניחו ששיטה זו קיימת ועובדת גם אם לא פתרם סעיף זה). מותר להקצות זיכרון נוסף על ה-heap כל עוד אתם מקצים כמות זיכרון ליניארית במספר הקוביות ($O(N)$ זיכרון) בלבד. מותר לכתוב מחלקות ושיטות נוספות ולהיעזר בהן במהלך הפתרון.
רמז: ניתן להיעזר במערך של N מחסניות להחזקת אבני הפירמידות.

67.101 - 67.130
12/10/21

שאלה 4 (33 נקודות)

בשאלה זו אין להשתמש ברקורסיה!

הערך המירבי שיכול להחזיק משתנה מטיפוס `int` ב-Java מוגבל בגודלו. בשאלה זו, עליכם לכתוב מחלקה בשם `BigInt` המייצגת מספר שלם אי-שלילי בגודל לא מוגבל. מחלקה זו צריכה לממש את השיטות הבאות:

<code>public BigInt(String s)</code>	בונה <code>BigInt</code> חדש המייצג מספר הנתון ע"י מחרוזת (5 נק').
<code>public static BigInt add(BigInt a, BigInt b)</code>	מחזירה <code>BigInt</code> חדש שהוא הסכום של <code>a</code> ו- <code>b</code> . (שימו לב שמתודה זו היא סטטית.) (12 נק')
<code>public int compareTo(BigInt other)</code>	משווה <code>BigInt</code> זה ל- <code>other</code> . מחזירה 1 אם <code>BigInt</code> זה יותר גדול, -1 אם <code>other</code> יותר גדול, או 0 אם הם שווים (12 נק').
<code>public String toString()</code>	מחזירה את ערך <code>BigInt</code> זה כמחרוזת (4 נק').

מותר להוסיף שיטות (פרטיות) נוספות.

מותר להניח כי הפרמטרים המועברים לכל השיטות אינם `null`. כמו כן, מותר להניח כי המחרוזות המועברות לבנאי היא חוקית (כלומר מייצגת מספר שלם אי-שלילי, ללא אפסים בהתחלה). המספר 0 מיוצג אך ורק ע"י המחרוזת "0".
הנה דוגמה לקטע תוכנית המשתמש במחלקה `BigInt`:

```
BigInt a = new BigInt("1");
BigInt b = new BigInt("12");
BigInt c = BigInt.add(a, b);
System.out.println("a = " + a.toString());
System.out.println("b = " + b.toString());
System.out.println("c = " + c.toString());
System.out.println(a.compareTo(b));
```

הפלט של קטע זה הוא:

```
a = 1
b = 12
c = 13
-1
```

67-101-67-130
נ"ח/ח"ס

לשם מימוש המחלקה BigInt, ניתן להשתמש במחלקה IntList, המייצגת רשימה מקושרת של ערכים מטיפוס int. (מחלקה זו נתונה ואינכם צריכים לממש אותה.) המחלקה IntList מספקת את הפעולות הבאות:

<code>public IntList()</code>	בונה רשימה ריקה.
<code>public void addFirst(int i)</code>	מוסיפה את הערך הנתון לראש הרשימה.
<code>public void addLast(int i)</code>	מוסיפה את הערך הנתון לסוף הרשימה.
<code>public int size()</code>	מחזירה את מספר הערכים הנמצאים ברשימה.
<code>public IntListIterator iterator()</code>	מחזירה איטרטור העובר על הרשימה ע"פ הסדר (מהראש לסוף).
<code>public IntListIterator reverseIterator()</code>	מחזירה איטרטור העובר על הרשימה בסדר הפוך (מהסוף לראש).

האובייקטים המוחזרים ע"י השיטות `iterator()` ו-`reverseIterator()` מממשים את הממשק `IntListIterator`, המגדיר את הפעולות הבאות:

<code>public boolean hasNext()</code>	מחזירה true אם נותרו ערכים לעבור עליהם, false אחרת.
<code>public int next()</code>	מחזירה את הערך הבא. קריאה לשיטה זו תגרור לשגיאה בתוכנית אם לא נותרו עוד ערכים.

לצורך המרה של מחרוזת למספר ניתן להשתמש בשיטות הבאות:

- השיטה הסטטית `Integer.parseInt(String s)` מחזירה ערך מטיפוס int המתאים למחרוזת הנתונה. לדוגמא, ערך הביטוי `Integer.parseInt("12")` הוא 12. (שימו לב כי המחרוזת חייבת לייצג מספר בתחום המותר עבור int.)
- השיטה `substring(int start, int end)` במחלקה `String` מחזירה את תת-המחרוזת המתחילה במקום start ומסתיימת במקום end-1. לדוגמא, ערך הביטוי `"smiles".substring(1,5)` הוא "mile".
- השיטה `length()` במחלקה `String` מחזירה את אורך המחרוזת. לדוגמא, ערך הביטוי `"smiles".length()` הוא 6.

רמז: ניתן לחשוב על מספר שלם בתור סדרה של ספרות (כל ספרה בין 0 ל-9).

ב ה צ ל ה !

